

Introduction to MongoDB

Documents and collections with Mongita

Gregory S. DeLozier, PhD

What Is a Document Database?

A record has **named fields and values**.

Values can include lists and nested records.

A collection holds related documents.

One Record, Several Kinds of Value

```
{  
  "name": "Casey",  
  "age": 3,  
  "vaccinations": ["rabies", "distemper"],  
  "details": {"color": "brown", "weight_kg": 18}  
}
```

Scalar, list, nested document.

The Database Can Read the Fields

```
pets.find({"age": {"$gt": 2}})
```

Ask for pets older than two.

Query the data inside the document.

Flexible Fields Still Need Meaning

Choice	Example
Required value	Every pet needs a name
Consistent type	Age is a number
Optional field	Weight may be absent
Shared units	Weight is recorded in kilograms

Flexibility leaves us with design decisions.

The Idea Predates MongoDB

Milestone	Idea
Lotus Notes, 1989	Documents and forms for shared work
Apache CouchDB, 2008	Document database becomes an Apache project
MongoDB, 2009	Launch of a new document server

MongoDB did not invent document databases.

MongoDB Arrives

2007: 10gen is founded.

2009: MongoDB launches.

Documents familiar to application programmers.

A server designed for growing applications.

A Common Reference Point

MongoDB's vocabulary travels:

collections, documents, filters, updates, ObjectIds.

Other products offer compatibility with its interface.

Related Interfaces, Different Systems

System	What it provides
MongoDB	The database server
Amazon DocumentDB	A service with MongoDB API compatibility
Mongita	A local Python library with a subset of the API
CouchDB	A document database with its own interface

Check the operations your application uses.

Sometimes Tables Are Constraining

Test	Glucose	Sodium	Body region	Report
Blood chemistry	92	140	-	-
X-ray	-	-	Left wrist	Text

One column for every possible result?

Keep the Fields That Apply

```
{"test": "Blood chemistry", "patient_id": "P17",  
  "results": {"glucose_mg_dl": 92,  
              "sodium_mmol_l": 140}}
```

```
{"test": "X-ray", "patient_id": "P17",  
  "results": {"body_region": "Left wrist",  
              "report": "Report text"}}
```

Different fields. Shared meaning for common fields.

A Purchase and Its Line Items

Record	Contains
Purchase	Receipt identifier, date
Purchase item	Purchase, SKU, quantity, price paid
Product	SKU, description, current price

A receipt normally needs all its line items.

Store the Items with the Purchase

```
purchase = {  
  "purchase_id": "R1042",  
  "items": [  
    {"sku": "MILK-1L", "quantity": 2,  
     "unit_price_cents": 249},  
    {"sku": "BREAD-W", "quantity": 1,  
     "unit_price_cents": 325},  
  ],  
}
```

One document includes the line items.

What Belongs Together?

Purchase-specific line items: **embed**.

Shared product catalog: **reference by SKU**.

What changes when the query is total sales for one SKU?

A Short Interactive Session

```
from lesson import connect
from bson.objectid import ObjectId
client, animals = connect()

result = animals.insert_one(
    {"name": "Casey", "type": "dog"})
id_text = str(result.inserted_id)
key = {"_id": ObjectId(id_text)}
print(animals.find_one(key))
```

Change It, Then Remove It

```
animals.update_one(key, {"$set": {"age": 3}})
print(animals.find_one(key))

animals.delete_one(key)
print(animals.find_one(key))  # None
client.close()
```

The same identifier selects the same document.

Back to database.py

App calls	Layer uses
<code>create_pet(data)</code>	<code>insert_one()</code>
<code>get_pet(id)</code>	<code>find_one()</code>
<code>update_pet(id, data)</code>	<code>update_one()</code> with <code>\$set</code>
<code>delete_pet(id)</code>	<code>delete_one()</code>

Keep the application's database functions.

IDs Are Now Strings in the App

```
# Database operation returns an ObjectId.  
result = pets_collection.insert_one(pet)  
  
# Application receives its text representation.  
return str(result.inserted_id)
```

Integer row ID becomes **24-character hexadecimal text**.

Convert Back for the Lookup

```
def get_pet(id):  
    object_id = _to_object_id(id, "pet id")  
    pet = pets_collection.find_one({"_id": object_id})  
    if pet is None:  
        return None  
    return pet_to_dict(pet)
```

String in. ObjectId for the query. String out.

Update Through the Layer

```
def update_pet(id, data):  
    object_id, _ = _require_existing_pet(id)  
    pet = _normalize_pet_data(data)  
    pets_collection.update_one(  
        {"_id": object_id}, {"$set": pet})
```

Validate the values, then save the fields.

Run the Pets Application

```
cd chapter-11-intro-to-mongo/code/pets-app  
python -m pip install -r requirements.txt  
flask --app app run --debug
```

Create an owner, then a pet. Follow its Update link.

Find the ObjectId string in the address.

What Changed?

Database operations and identifier conversion.

The app still calls **the database layer**.

Where would an owner check belong?